

Cloud Computing and Serverless Architecture

Evaluation Dataset Document for Retrieval-Augmented Question Answering

Purpose

This document is a self-contained academic-style knowledge source designed for testing document retrieval and Retrieval-Augmented Generation (RAG) systems. It contains related concepts, definitions, implementation details, comparisons, examples, trade-offs, and conclusions distributed across multiple sections so that retrieval methods can be evaluated on both exact terminology and semantic meaning.

Document scope

The document focuses on cloud computing, virtualization, containers, serverless computing, Function-as-a-Service (FaaS), managed databases, object storage, API gateways, event-driven systems, observability, security, cost management, scalability, and deployment patterns.

Suggested use in the Academic RAG QA System

Upload this PDF as a second evaluation document after the internship report. Build its chunks and FAISS index, then ask questions whose answers are distributed across different sections. The same questions can later be evaluated using Dense retrieval, BM25, Hybrid retrieval, and Hybrid plus Cross-Encoder reranking.

Evaluation note: The document deliberately contains terminology that sometimes appears in several sections. This allows experiments to distinguish exact keyword matching from semantic retrieval and second-stage relevance ranking.

1. Foundations of Cloud Computing

Cloud computing is a model for delivering computing resources over a network on demand. Instead of purchasing and maintaining all physical infrastructure locally, an organization can obtain compute, storage, networking, databases, and other services from a cloud provider. A major characteristic of cloud computing is resource pooling: physical infrastructure is shared among many customers while logical isolation is maintained.

Elasticity describes the ability to increase or decrease resources as workload changes. Scalability is related but is often used more broadly to describe the capacity of a system to handle additional users, requests, or data. Vertical scaling increases the capacity of an existing resource, while horizontal scaling adds more instances or service replicas.

1.1 Service Models

Infrastructure as a Service (IaaS) provides virtual machines, networks, and storage. The customer generally manages the operating system and application stack. **Platform as a Service (PaaS)** abstracts much of the infrastructure and provides a managed application platform. **Software as a Service (SaaS)** delivers a complete application to end users.

1.2 Deployment Models

A public cloud provides resources through a cloud provider's shared infrastructure. A private cloud is dedicated to one organization. A hybrid cloud combines private and public environments and may be used when workloads or data have different placement requirements.

1.3 Virtualization and Containers

Virtualization uses a hypervisor to create virtual machines that behave like independent computers. Each virtual machine can run its own operating system. Containers provide a lighter form of isolation by sharing the host operating-system kernel while packaging an application with its dependencies. Containers generally start faster and use fewer resources than full virtual machines, although they provide a different isolation boundary.

For distributed applications, containers can simplify packaging and deployment. However, operating a container platform still requires decisions about scheduling, networking, scaling, health checks, and updates.

2. Serverless Computing and Function-as-a-Service

Serverless computing is a cloud execution model in which the provider manages the underlying servers and much of the operational infrastructure. The term does not mean that servers do not exist; rather, application developers normally do not manage those servers directly. A common serverless model is Function-as-a-Service (FaaS), where application logic is divided into functions that execute in response to events or requests.

2.1 Function Execution

A serverless function commonly receives an event, performs a bounded unit of work, and returns a result or produces a side effect. Events may come from an HTTP request, a message queue, an object upload, a scheduled rule, or a database change. Function execution is generally stateless from the perspective of an individual invocation, so durable application state is usually stored in external services such as databases or object storage.

2.2 Cold Starts and Warm Invocations

A cold start occurs when the provider must initialize an execution environment before invoking a function. Initialization can include loading the runtime, dependencies, and application code. A warm invocation reuses an already prepared environment and can therefore have lower startup latency. Cold-start behavior depends on runtime, package size, initialization work, traffic patterns, and provider implementation.

2.3 Stateless Design

Stateless design is important in serverless systems because individual invocations should not depend on local memory remaining available for a future request. A contact form, for example, can send an HTTP request to an API gateway, invoke a function, validate the input, and store the submission in a managed database. The database provides persistence independently of the function execution environment.

2.4 Benefits and Limitations

Serverless architectures can reduce infrastructure management, support automatic scaling, and allow teams to pay according to service usage. They can be useful for event-driven workloads, APIs, scheduled jobs, and lightweight data-processing tasks. Limitations include cold-start latency, execution-duration limits, platform-specific behavior, distributed debugging, and the need to understand service-specific pricing.

3. Architecture of a Serverless Web Application

A typical serverless web application separates the user interface, request routing, business logic, and persistent storage. A browser may download static HTML, CSS, and JavaScript from object storage or a content delivery network. API requests can then pass through an API gateway to serverless functions, which communicate with managed data services.

3.1 Request Flow

Consider a serverless contact application. The browser displays a form containing a name, email address, and message. When the user submits the form, the frontend sends an HTTPS request to an API endpoint. The API gateway validates basic request routing and invokes a backend function. The function validates the payload, adds metadata such as a timestamp, and writes the record to a managed database.

A successful response travels back through the API gateway to the browser. If validation fails, the function can return a client error. If a downstream service fails, the application should return a controlled error response and record diagnostic information for operators.

3.2 Static Frontend and Object Storage

Object storage is well suited to static assets because files such as HTML, CSS, JavaScript, images, and documents can be stored independently of application servers. A static frontend can therefore be deployed without maintaining a traditional web server for each application.

3.3 API Gateway

An API gateway provides an entry point for client requests. It can route HTTP methods to backend functions and can support authentication, throttling, request transformation, monitoring, and other API-management capabilities. Separating the API layer from the function implementation also allows the backend to evolve without requiring the frontend to know its internal execution model.

3.4 Managed Databases

A serverless application can use either relational or NoSQL databases depending on its data model. Relational databases are useful when structured records and relationships require SQL transactions. NoSQL databases can be useful for key-value or document-oriented access patterns and large-scale request workloads. The correct choice depends on access patterns, consistency requirements, transaction needs, and operational constraints.

4. Event-Driven Systems and Data Processing

Event-driven architecture organizes application behavior around events. An event represents a fact that something happened, such as a file being uploaded, a payment being completed, a record being created, or a message entering a queue. Producers generate events and consumers react to them. This model can reduce direct coupling between components.

4.1 Queues and Asynchronous Processing

A message queue can temporarily hold work between a producer and a consumer. This separation is useful when processing speed differs between the two components. For example, a web application can place an image-processing task into a queue and immediately return a response to the user. A worker or serverless function can process the task asynchronously.

Queues can also improve resilience. If a consumer becomes temporarily unavailable, messages can remain in the queue until processing resumes, subject to retention and failure policies. Dead-letter queues can isolate messages that repeatedly fail so that they can be investigated without blocking other work.

4.2 Object-Triggered Processing

Object storage can act as an event source. When a new PDF, image, or data file is uploaded, an event can invoke a processing function. The function may extract metadata, validate the file, transform the content, and store the result in another service. This pattern is useful for document processing pipelines because storage and compute remain loosely coupled.

4.3 Scheduled Workloads

Not all event-driven tasks originate from users. A scheduler can invoke a function at regular intervals for tasks such as generating reports, cleaning temporary records, checking system conditions, or refreshing cached information. Scheduled functions should be designed to tolerate retries because distributed systems can deliver an event more than once.

4.4 Idempotency

Idempotency means that processing the same request or event multiple times does not produce an incorrect cumulative effect. It is especially important in event-driven systems because retries can occur after timeouts or transient failures. A common approach is to assign a unique event identifier and record which events have already been processed.

5. Security, Reliability, and Observability

Cloud applications should apply security controls at multiple layers rather than relying on a single protection mechanism. Identity and access management determines which users, services, and functions are allowed to perform particular actions. The principle of least privilege means granting only the permissions required for a task.

5.1 Authentication and Authorization

Authentication verifies who or what is making a request, while authorization determines what that identity is allowed to do. API endpoints can use tokens, managed identity mechanisms, or other authentication systems. Authorization policies should restrict access to only the required resources and operations.

5.2 Data Protection

Data should be protected both in transit and at rest. HTTPS protects network communication between clients and APIs. Managed storage and databases commonly provide encryption capabilities for stored data. Secrets such as API keys and database credentials should not be embedded in frontend source code or committed to public repositories.

5.3 Reliability

Reliable systems anticipate failures rather than assuming every dependency is always available. Timeouts, retries with appropriate backoff, validation, controlled error handling, and durable queues can reduce the impact of transient failures. Retries must be designed carefully because blindly repeating a non-idempotent operation can create duplicate effects.

5.4 Observability

Observability combines logs, metrics, and traces to help engineers understand system behavior. Logs record events and diagnostic information. Metrics provide numerical measurements such as request count, latency, error rate, and function duration. Traces help connect activity across multiple services. Together, these signals can help identify slow functions, failing APIs, and unexpected workload patterns.

For a serverless application, useful operational measurements include invocation count, duration, cold-start frequency when available, error count, throttling, database latency, queue depth, and API response latency.

6. Cost, Scalability, and Architectural Trade-offs

Cloud architecture decisions should consider performance, reliability, security, operational effort, and cost together. A design that minimizes infrastructure management may have different cost behavior from a design based on continuously running virtual machines.

6.1 Serverless Cost Model

Serverless services commonly charge according to dimensions such as request count, execution duration, memory allocation, data transfer, storage, or related service usage. This can make serverless attractive for workloads with variable traffic because resources are not necessarily running continuously at a fixed capacity.

However, serverless does not automatically mean inexpensive. High-volume workloads, inefficient functions, excessive logging, large data transfers, or expensive downstream services can increase cost. Cost analysis should therefore examine the complete request path rather than focusing only on the function price.

6.2 Scalability

Serverless platforms can automatically create additional execution environments as demand grows, subject to concurrency limits and service quotas. Horizontal scaling is therefore often built into the execution model.

Downstream systems must still be capable of handling the increased load. A rapidly scaling function can overwhelm a database if connection management and capacity planning are ignored.

6.3 Serverless versus Virtual Machines

Virtual machines provide greater control over the operating system, installed software, networking, and long-running processes. They can be appropriate when an application requires persistent processes, custom operating-system configuration, specialized software, or predictable long-running workloads.

Serverless functions reduce infrastructure administration and are particularly convenient for short-lived event-driven tasks. The trade-off is less control over the execution environment and potential platform-specific constraints.

6.4 Serverless versus Containers

Containers provide application portability and consistent packaging while retaining more control over the runtime than many serverless function platforms. Container orchestration, however, introduces operational concepts such as clusters, scheduling, service discovery, and deployment management. Serverless functions can remove much of this infrastructure responsibility when the application fits the function execution model.

7. Design Example and Conclusion

A small academic document-management application can combine the concepts described in this document. A user uploads a PDF to object storage. The upload event invokes a processing function, which validates the file and extracts metadata. The extracted content can be stored for later retrieval. A web frontend communicates with an API gateway when the user asks a question. A backend function retrieves relevant passages and generates a response using an AI service.

The architecture can be represented as: Browser → Static Object Storage → API Gateway → Serverless Function → Managed Database or Search Service. For asynchronous document processing, the upload event can be routed through a queue before the processing function. Monitoring services collect logs and metrics from the API, functions, and data layer.

7.1 Key Design Decisions

The frontend should remain independent of backend implementation details. The API should validate requests and return consistent response formats. Functions should remain focused on individual tasks rather than becoming a single large application. Persistent state should be stored in durable services. Sensitive credentials should be protected using an appropriate secret-management mechanism. Monitoring should be included from the beginning rather than added only after failures occur.

7.2 Practical Decision Guide

Requirement	Potential approach	Reason
Static website	Object storage / CDN	Simple delivery of static assets
HTTP API	API gateway + function	Managed request routing and execution
Short event task	Serverless function	Automatic execution and scaling
Long-running process	VM or container	Better control over persistent execution
Asynchronous workload	Queue + worker/function	Decouples producer and consumer
Structured relational data	Managed SQL database	Transactions and relational queries
Key-value/document access	Managed NoSQL database	Flexible high-scale access patterns
Operational diagnosis	Logs + metrics + traces	Visibility across distributed services

Conclusion

Cloud computing provides flexible access to infrastructure and managed services, while serverless computing further abstracts infrastructure management for event-driven and short-lived workloads. Effective designs balance scalability, reliability, security, observability, performance, and cost. The appropriate architecture depends on workload characteristics and operational requirements rather than on a single universally correct model.